Design Note
Issue 0B, June 2026

Anti-Aliasing Filter for a 12-bit ADC Input

Overview

This note covers the design of the anti-aliasing filter that sits between the sensor amplifier and the 12-bit ADC input of the microcontroller. It sets out the requirements, the choice of a second-order Sallen-Key low-pass filter, the component values and the predicted performance across component tolerance, along with the firmware sampling arrangement and the tests planned for the first boards.

Key Figure

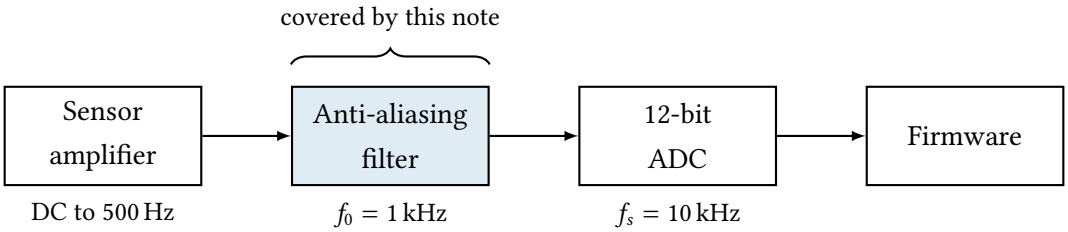


Figure 1: The sensor channel, with the stage this note covers shaded.

Revision Control

Issue	Release Date	Description of Changes
Issue 0A	March 2026	Initial issue for review.
Issue 0B	June 2026	C_1 changed to two 10 nF parts in parallel, so that both capacitor positions use the same component. Tolerance analysis added.

Table 1: Document Revision Control

Contents

Revision Control **I**

List of Figures **III**

List of Tables **III**

Nomenclature **IV**

1 Introduction **1**
1.1 Scope 1

2 Requirements **1**
2.1 Where the stopband figure comes from 1

3 Design **2**
3.1 Choice of circuit 2
3.2 Design equations 2
3.3 Component values 3

4 Predicted Performance **4**
4.1 Frequency response 4
4.2 Effect of component tolerance 4
4.3 Limits of this analysis 5

5 Firmware Interface **6**
5.1 Sampling sequence 6
5.2 Example code 6

6 Verification Plan **8**
6.1 Open items 8

References **9**

A Derivation of the Transfer Function **10**

List of Figures

1 The sensor channel, with the stage this note covers shaded. 1
2 The unity-gain Sallen-Key low-pass filter. C_1 is the feedback capacitor and C_2 is the capacitor to ground. 2
3 Calculated magnitude response at the nominal component values. The curve passes 4.2 dB below the stopband requirement. 4
4 The sampling sequence over two sample periods. The timer starts each conversion, and the interrupt is raised when the result is ready. 6

List of Tables

1 Document Revision Control I
2 Requirements for the anti-aliasing filter. 1
3 Component values for the filter. 3
4 Predicted performance at the nominal values and at the worst combination of component tolerances. 5
5 Tests planned for the first boards. 8

Nomenclature

f_0	Natural frequency of the filter
f_s	Sample rate
Q	Quality factor of the filter
ADC	Analogue-to-Digital Converter
dBFS	Decibels relative to the full-scale input of the converter
GBW	Gain-bandwidth product of an operational amplifier
LSB	Least significant bit, the smallest step the converter can resolve

1 Introduction

THE sensor channel on this board is sampled by the microcontroller's 12-bit ADC at 10 kHz. Any signal or interference above half the sample rate is not removed by sampling, it is folded back into the wanted band where it cannot be told apart from the real signal, so it has to be filtered out before the converter and not afterwards in firmware [1].

This note records the design of that filter. It covers the requirements the filter has to meet in section 2, the circuit and the choice of component values in section 3, the performance predicted from those values across component tolerance in section 4, how the firmware samples the filtered signal in section 5, and the tests planned for the first boards in section 6.

1.1 Scope

The note covers the filter stage only, from the output of the sensor amplifier to the ADC pin, as shown in fig. 1 on the title page. The sensor amplifier and the digital processing that follows the converter are covered in their own documents. All of the performance figures here are calculated, as no boards have been built at this issue, and section 6 lists what will be measured to confirm them.

2 Requirements

The requirements for the filter are listed in table 2. They come from the sensor, which produces signals from DC up to 500 Hz, and from the converter, which samples at 10 kHz.

Ref.	Requirement	Value
R1	Supply	Single 3.3 V rail, shared with the microcontroller
R2	Signal band	DC to 500 Hz
R3	Sample rate	10 kHz, fixed by the firmware
R4	Passband gain	0 dB, flat to within 1 dB over the signal band
R5	Stopband	At least 35 dB of attenuation at 9.5 kHz
R6	Parts	One op-amp, with resistors and capacitors from standard value series

Table 2: Requirements for the anti-aliasing filter.

2.1 Where the stopband figure comes from

With a sample rate f_s of 10 kHz, the frequencies that fold back onto the signal band of 0 to 500 Hz are those between 9.5 kHz and 10.5 kHz. A low-pass filter attenuates the bottom of that range the least, so 9.5 kHz is the frequency the requirement is set at.

An ideal 12-bit converter has a signal-to-noise ratio of

$$\mathrm{SNR} = 6.02N + 1.76 = 74\,\mathrm{dB} \quad \text{for } N = 12, \tag{1}$$

so an aliased signal does no harm once it is more than 74 dB below full scale, as it is then smaller than the converter's own quantisation noise. The interference expected at the filter input is no larger than −40 dBFS, which means 35 dB of attenuation takes it to −75 dBFS, just under that floor. A larger figure would need a higher-order filter and a second op-amp, which requirement R6 rules out.

3 Design

3.1 Choice of circuit

A single resistor and capacitor would be the simplest filter, but its response falls at only 20 dB per decade, so a corner frequency low enough to give 35 dB at 9.5 kHz would sit at around 170 Hz and would cut into the signal band. A second-order filter falls at 40 dB per decade, which allows the corner to be placed at 1 kHz, comfortably above the signal band, whilst still meeting the stopband requirement.

The unity-gain Sallen-Key circuit [2] in fig. 2 was chosen because it needs only one op-amp, two resistors and two capacitors, and because with the op-amp connected as a follower the passband gain is exactly one and does not depend on any resistor ratio. A Butterworth response is used, as it is the flattest in the passband and requirement R4 is about flatness.

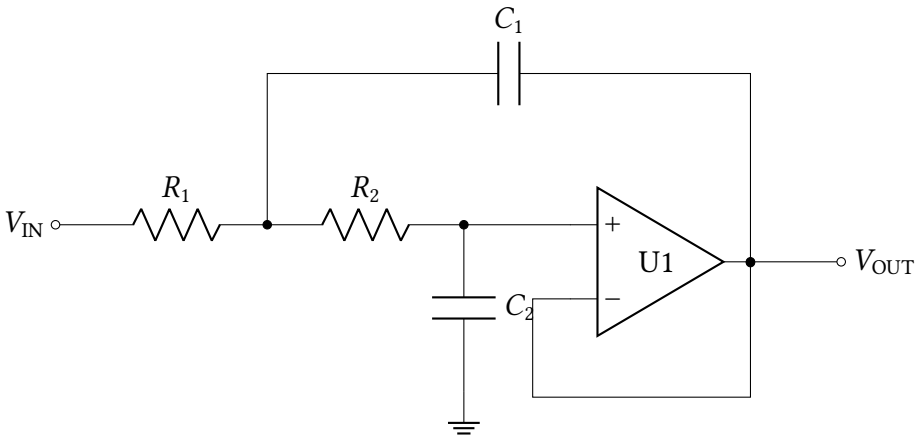


Figure 2: The unity-gain Sallen-Key low-pass filter. C_1 is the feedback capacitor and C_2 is the capacitor to ground.

3.2 Design equations

The transfer function of the circuit, which is derived in appendix A, is

$$H(s) = \frac{V_{\mathrm{OUT}}}{V_{\mathrm{IN}}} = \frac{1}{1 + s\,C_2(R_1 + R_2) + s^2 R_1 R_2 C_1 C_2}. \tag{2}$$

Comparing this with the standard second-order form gives the natural frequency and the quality factor,

$$f_0 = \frac{1}{2\pi\sqrt{R_1 R_2 C_1 C_2}}, \tag{3}$$

$$Q = \frac{\sqrt{R_1 R_2 C_1 C_2}}{C_2(R_1 + R_2)}. \tag{4}$$

If the two resistors are made equal, so that $R_1 = R_2 = R$, eq. (4) reduces to $Q = \tfrac{1}{2}\sqrt{C_1/C_2}$, and the quality factor then depends only on the ratio of the capacitors. A Butterworth response needs $Q = 1/\sqrt{2}$, which means $C_1 = 2C_2$.

3.3 Component values

Capacitors come in far fewer values than resistors, so the capacitors are chosen first. Taking C_2 as 10 nF, C_1 has to be 20 nF, which is made from two 10 nF parts in parallel. This gives the exact two-to-one ratio, and it means all three capacitors are the same component, so they will tend to drift together with temperature and the ratio that sets Q is preserved.

Rearranging eq. (3) for a corner at 1 kHz gives the resistor value,

$$R = \frac{1}{2\pi f_0 \sqrt{C_1 C_2}} = \frac{1}{2\pi \times 1\,\mathrm{kHz} \times 14.14\,\mathrm{nF}} = 11.25\,\mathrm{k\Omega}, \tag{5}$$

and the nearest standard value is 11.3 kΩ, which moves the corner to 996 Hz. The full list of parts is in table 3.

Part	Value	Tolerance	Notes
R_1, R_2	11.3 kΩ	1 %	Thin film, E96 series
C_1	20 nF	5 %	Two 10 nF C0G parts in parallel
C_2	10 nF	5 %	C0G, the same part as C_1
U1	Op-amp	n/a	Rail-to-rail input and output, GBW of 1 MHz or more

Table 3: Component values for the filter.

C0G capacitors are specified because their value does not change with the voltage across them, whereas the capacitance of the cheaper X7R type falls as the voltage rises, which would distort the signal as well as moving the corner frequency. The op-amp needs rail-to-rail inputs and outputs because it runs as a follower from a single 3.3 V rail, and a gain-bandwidth of at least a hundred times the corner frequency keeps its own roll-off from affecting the response [3]. A general-purpose CMOS part such as the MCP6002 meets both points.

4 Predicted Performance

4.1 Frequency response

Substituting $s = \mathrm{j}2\pi f$ into eq. (2) gives the magnitude of the response at any frequency,

$$|H(f)| = \frac{1}{\sqrt{\left(1 - (f/f_0)^2\right)^2 + \left(f/(Q f_0)\right)^2}}, \tag{6}$$

which is plotted in fig. 3 for the nominal component values. The response is 0.27 dB down at the top of the signal band and 39.2 dB down at 9.5 kHz, so both requirements are met with the nominal values.

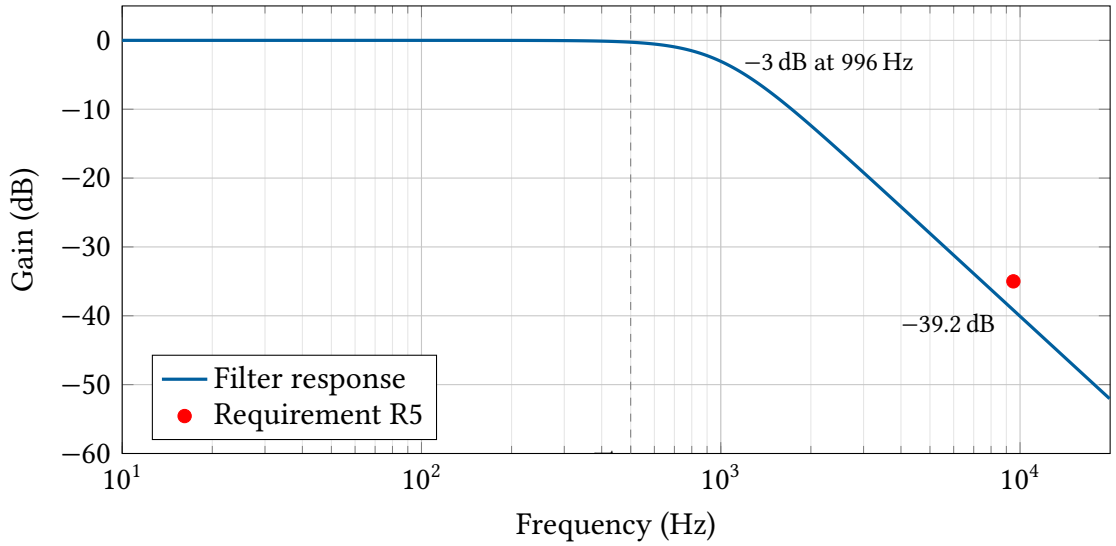


Figure 3: Calculated magnitude response at the nominal component values. The curve passes 4.2 dB below the stopband requirement.

4.2 Effect of component tolerance

The nominal figures only show that the design is sound on paper, as every board will be built with parts that are a little off their marked values. The response was therefore recalculated with each of the four values at either end of its tolerance band, which gives sixteen combinations, and the worst result from those is shown against the requirement in table 4.

The stopband requirement is met with 3.1 dB in hand in the worst case, and the passband stays within half a decibel. The natural frequency moves by about six per cent either way, almost all of which is due to the capacitors, so tighter resistors would gain very little.

Taking every part to its limit at once is pessimistic, so the calculation was repeated as a Monte Carlo run of 20,000 builds, with each value drawn at random from within its tolerance band. The standard deviation of f_0 was 21 Hz, and no build fell outside the worst-case figures in table 4.

Quantity	Requirement	Nominal	Worst case
Natural frequency, f_0	n/a	996 Hz	939 Hz to 1059 Hz
Quality factor, Q	n/a	0.707	0.673 to 0.743
Gain at 500 Hz	Within 1 dB	−0.27 dB	−0.49 dB
Peaking in the passband	Within 1 dB	None	0.04 dB
Attenuation at 9.5 kHz	35 dB or more	39.2 dB	38.1 dB

Table 4: Predicted performance at the nominal values and at the worst combination of component tolerances.

4.3 Limits of this analysis

These figures assume an ideal op-amp. In a real Sallen-Key filter the attenuation does not keep increasing with frequency, because at high frequencies the op-amp's output impedance rises and the input signal reaches the output through C_1 [3]. With the part specified this is expected to matter only well above 10 kHz, but it cannot be confirmed by calculation alone, so it is one of the things the tests in section 6 are there to check.

5 Firmware Interface

The filter is designed around a sample rate of exactly 10 kHz, so the firmware has to hold that rate steady. If conversions were started from the main loop, the interval between them would vary with whatever else the processor was doing, and that jitter would add noise to the samples. A hardware timer is used to start each conversion instead, which means the sample instants are set by the timer and not by the code.

5.1 Sampling sequence

The sequence for each sample is shown in fig. 4. The timer overflows every 100 µs and triggers the converter, which holds the input for its acquisition time and then converts. When the conversion finishes the converter raises an interrupt, and the interrupt routine stores the result.

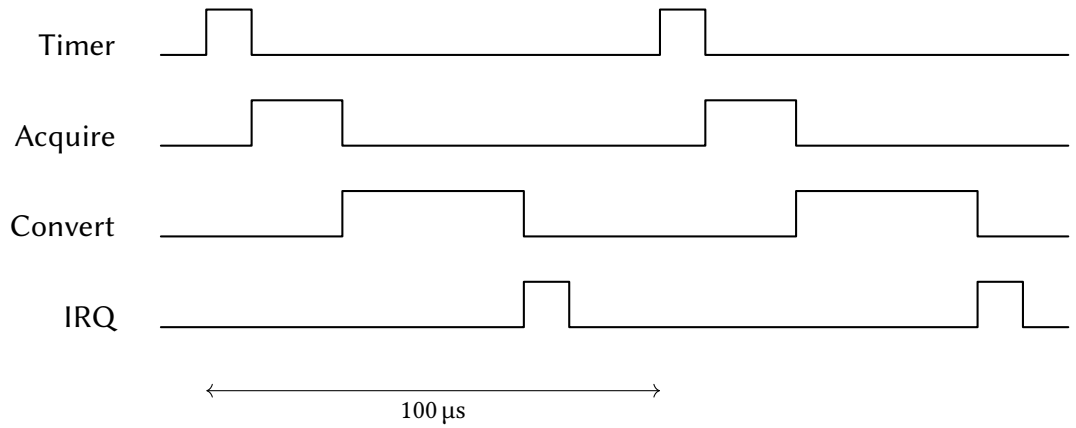


Figure 4: The sampling sequence over two sample periods. The timer starts each conversion, and the interrupt is raised when the result is ready.

The filter's output impedance is low, as it is driven by the op-amp, so the converter's sample capacitor settles well within the acquisition time and no extra buffer is needed between the two.

5.2 Example code

Listing 1 shows the arrangement. The function names stand in for those of the microcontroller's own hardware library, which differ from one family to the next, but the structure is the same on any of them.

```
#define SAMPLE_RATE_HZ  10000u
#define BUFFER_LENGTH   256u

static volatile uint16_t buffer[BUFFER_LENGTH];
static volatile uint16_t head;
```

```

/* Called once at start-up. The timer starts a conversion every
   100 us, so the sample rate does not depend on the main loop. */
void sampling_init(void)
{
    adc_select_channel(ADC_CHANNEL_SENSOR);
    adc_set_trigger(ADC_TRIGGER_TIMER);
    adc_enable_interrupt();

    timer_set_period_us(1000000u / SAMPLE_RATE_HZ);
    timer_start();
}

/* End-of-conversion interrupt: store the result and move on. */
void adc_isr(void)
{
    buffer[head] = adc_read_result();
    head = (head + 1u) % BUFFER_LENGTH;
}
```

Listing 1: Timer-triggered sampling into a ring buffer.

The interrupt routine only stores the sample and moves the index on. Anything slower, such as scaling or further digital filtering, is left to the main loop, so that the routine is always finished long before the next conversion completes.

6 Verification Plan

No boards have been built at this issue, so this section lists the tests that will be carried out on the first ones, and the result each has to give for the design to pass. The measured results will be added at the next issue.

Test	Method	Pass criterion
T1	Apply DC levels of 0.3 V, 1.65 V and 3.0 V to the input and measure the output.	Output within 1 % of the input at each level.
T2	Apply a 100 mV rms sine wave on a 1.65 V offset, and step it from 50 Hz to 500 Hz.	Gain within 1 dB of the DC gain at every step (requirement R4).
T3	Raise the same sine wave to 1 V rms and measure the gain at 9.5 kHz, then at 20 kHz, 50 kHz and 100 kHz.	At least 35 dB of attenuation at 9.5 kHz (requirement R5), and no less than that at the higher frequencies.
T4	With the firmware running, toggle a spare pin in the interrupt routine and measure the period on an oscilloscope.	A period of 100 µs, with no visible jitter at 1 µs per division.

Table 5: Tests planned for the first boards.

Test T3 is taken beyond 9.5 kHz to find the point where the attenuation stops increasing, as described at the end of section 4. If it turns out to fall below 35 dB within the range tested, the fix would be an op-amp with a wider bandwidth, which is a change of part and not of the board.

6.1 Open items

- The −40 dBFS limit on interference at the filter input, used in section 2, is an estimate. It should be measured on the first assembled unit, in its enclosure.
- The op-amp is specified by its parameters and not yet by a part number, which is to be fixed once stock and price have been checked.

References

[1] P. Horowitz and W. Hill, *The Art of Electronics*, 3rd ed. Cambridge: Cambridge University Press, 2015.

[2] R. P. Sallen and E. L. Key, "A practical method of designing RC active filters," *IRE Transactions on Circuit Theory*, vol. 2, no. 1, pp. 74–85, Mar. 1955.

[3] J. Karki, "Analysis of the Sallen-Key architecture," Texas Instruments, Application Report SLOA024B, 2002.

A Derivation of the Transfer Function

This appendix derives eq. (2) for the circuit in fig. 2. Let V_A be the voltage at the junction of R_1 and R_2. The op-amp is connected as a follower, so the voltage at its non-inverting input is equal to V_{OUT}, and no current flows into that input.

The current through R_2 can therefore only flow into C_2, which gives

$$\frac{V_A - V_{\mathrm{OUT}}}{R_2} = sC_2V_{\mathrm{OUT}} \quad \Longrightarrow \quad V_A = V_{\mathrm{OUT}}\,(1 + sR_2C_2). \tag{7}$$

At the junction, the current arriving through R_1 leaves through R_2 and through C_1, whose other end is at V_{OUT}:

$$\frac{V_{\mathrm{IN}} - V_A}{R_1} = \frac{V_A - V_{\mathrm{OUT}}}{R_2} + sC_1(V_A - V_{\mathrm{OUT}}). \tag{8}$$

From the first result, $V_A - V_{\mathrm{OUT}} = sR_2C_2V_{\mathrm{OUT}}$. Substituting this into the second equation and multiplying through by R_1 gives

$$V_{\mathrm{IN}} - V_A = V_{\mathrm{OUT}}\left(sR_1C_2 + s^2R_1R_2C_1C_2\right), \tag{9}$$

and replacing V_A once more leaves

$$V_{\mathrm{IN}} = V_{\mathrm{OUT}}\left(1 + sC_2(R_1 + R_2) + s^2R_1R_2C_1C_2\right), \tag{10}$$

which rearranges to eq. (2). Setting the coefficients equal to those of the standard form, $1 + s/(Q\omega_0) + s^2/\omega_0^2$, gives eq. (3) and eq. (4).